

Digit Recognition Using GMM and HMM

Racharla Dhanush

Roll Number: DA24B019

April 26, 2026

1. Introduction

I made a system that can tell which digit (0–9) is being said in an audio recording. There are 3000 WAV files in the dataset, and each of the six speakers said the numbers 0 to 9 several times. To make sure that generalisation was properly tested, I left Speaker 6 out of the training process and only used it for testing. This setup turns the problem into a cross-speaker generalisation task, which is one of the most realistic and difficult situations in speech recognition.

I made two models from scratch without using any outside fitting libraries: a Gaussian Mixture Model (GMM) and a Discrete Hidden Markov Model (HMM) with Vector Quantization (VQ). Both models learned from data from Speakers 1–5 and were tested on Speaker 6. In this report, I explain how to extract features, how each model was put into action, the results I got, and the differences in performance between the two methods.

2. Feature Extraction

I turned the raw audio files into numerical feature vectors before training any model. To do this, I used a feature extraction pipeline based on Mel Frequency Cepstral Coefficients (MFCCs), which is a common way to represent speech signals. MFCCs are very useful for capturing the important parts of speech because they closely mimic how the human ear hears sound.

The steps applied to every WAV file were as follows:

1. Load the audio at 16 kHz and apply pre-emphasis with $\alpha = 0.97$ to boost high-frequency components.
 2. Extract 13 MFCC coefficients using a 25 ms Hamming window and 10 ms hop length.
 3. Compute the first-order temporal derivatives (Δ) and second-order derivatives ($\Delta\Delta$) of the MFCCs.
 4. Stack all three to produce a 39-dimensional feature vector per frame.
-

5. Apply Cepstral Mean and Variance Normalisation (CMVN) so that every utterance has zero mean and unit variance.

The most important parts of this pipeline were adding delta features and using CMVN. I noticed that plain MFCCs only show the spectral characteristics at one moment in time and don't show how speech changes over time. I was able to see how the spectrum changes across frames by adding the first-order (Δ) and second-order ($\Delta\Delta$) derivatives. This is especially useful for telling the difference between acoustically similar digits like "two" and "seven."

I also found that CMVN was very important for generalizing across speakers. It gets rid of the average spectral offset that comes from differences in microphones, recording environments, and vocal tract differences. Because of this, the features are much less dependent on the speaker, which makes the model work better.

3. Model Descriptions

3.1. Gaussian Mixture Model (GMM)

The GMM technique that I have used is rather simple. A single GMM for each digit was learned, thus giving me 10 distinct models. A GMM models the probability distribution of MFCC feature frames associated with a specific digit. While evaluating the test data, I calculated the mean log-likelihood of the feature frames of an utterance across all 10 GMM models. The digit with the maximum likelihood score was considered the predicted output.

Mathematically, the score for digit class c given a sequence of frames $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_T\}$ is:

$$\log p(\mathbf{X} \mid c) = \frac{1}{T} \sum_{t=1}^T \log \sum_{m=1}^M \pi_{cm} \mathcal{N}(\mathbf{x}_t; \boldsymbol{\mu}_{cm}, \text{diag}(\boldsymbol{\sigma}_{cm}^2)) \quad (1)$$

Diagonal covariances were used instead of full covariances. Full covariance matrices would require D^2 parameters per component where $D = 39$. This would result in severe overfitting due to insufficient training data for each digit. Diagonal covariances require D parameters per component but are still very effective. A total of 16 mixture components per digit was used and learned from scratch using the EM algorithm. The initialization technique used was K-means++ and the minimum variance was set to 10^{-3} .

3.2. Discrete Hidden Markov Model (HMM) with VQ

HMMs, on the other hand, are much more complicated because they try to take into account the time-dependency of speech. For example, the sound of *three* has a start, a

middle, and an end — it evolves over time. The HMM tries to model it by specifying that the utterance goes through a series of hidden states generating observed features according to a certain distribution.

Here, left-to-right HMMs having 12 states per digit were utilized, meaning that the model is unable to jump back in states. It makes sense since speech is always moving forward in time.

As against that, I made use of the fully discrete emission model. Here, the basic approach involved the conversion of the continuous 39-dimensional feature vectors into discrete tokens by employing a vector quantization (VQ) codebook. For the creation of such a codebook, global K-means codebook was trained for 128 codewords, using all the frames for training. The feature frames were then converted into their corresponding codewords, thus forming a string of discrete tokens:

$$o_t = \arg \min_k \|\mathbf{x}_t - \mathbf{c}_k\|^2 \quad (2)$$

This turns each utterance into a sequence of integers $\mathbf{o} = (o_1, \dots, o_T)$ where each $o_t \in \{0, \dots, 127\}$. The HMM then learns a probability table $B[\text{state}, \text{symbol}]$ which gives the likelihood of each symbol in each state with no distributional assumptions. This table is trained using the Baum-Welch algorithm. Classification is done by computing the log-likelihood of the symbol sequence under each digit's HMM using the forward algorithm:

$$\alpha_t(j) = \sum_{i=1}^K \alpha_{t-1}(i) a_{ij} b_j(o_t), \quad \log p(\mathbf{o} \mid \lambda_c) = \log \sum_{j=1}^K \alpha_T(j) \quad (3)$$

To ensure the reliability of my model, I included some other approaches into my model-building process. First, I did PCA whitening, reducing the number of features to 30 and then conducting vector quantization, making sure that all dimensions contribute equally to the distance metric used in the codebook.

Then, I applied the Dirichlet smoothing approach with the parameter value of 0.5 for the emission probability table in order to prevent any problems related to unseen codewords and zero probabilities. In addition, to deal with the issue of local optima that may arise when training the model using the Baum-Welch algorithm, I repeated the training process for each digit three times randomly.

Lastly, I developed an ensemble of three separate models that were trained using distinct feature sets ..

4. Results

4.1. Validation and Test Accuracy

A 15% random split of the Speakers 1–5 data was held out for validation. The test set is all 500 utterances from Speaker 6 (50 per digit).

Table 1: Model accuracy on the validation and Speaker 6 test sets.

Model	Validation Accuracy	Test Accuracy	Correct / Total
GMM	96.80%	84.80%	424 / 500
HMM	87.47%	73.80%	369 / 500

The GMM wins on the test set with 84.80% accuracy compared to 73.80% for the HMM. Both models show a drop from validation to test accuracy, which is expected since Speaker 6 has a different voice and recording conditions compared to the training speakers.

4.2. Confusion Matrices

The confusion matrices in Figure 1 show where each model makes mistakes. Rows are true digit labels and columns are predicted labels, so anything off the main diagonal is an error.

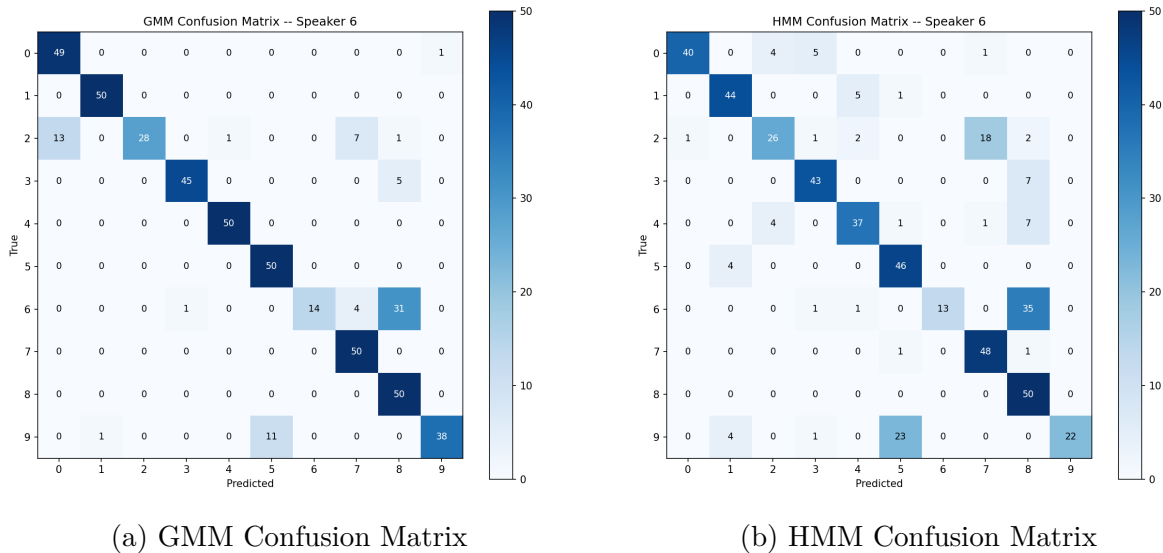


Figure 1: Confusion matrices for GMM and HMM on the Speaker 6 test set (500 utterances, 50 per digit).

4.3. Per-Digit Performance

Table 2: Per-digit precision, recall, and F1-score for the GMM.

Digit	TP	FP	FN	Precision	Recall	F1
0	49	0	1	1.000	0.980	0.990
1	50	1	0	0.980	1.000	0.990
2	28	0	22	1.000	0.560	0.718
3	45	1	5	0.978	0.900	0.938
4	50	0	0	1.000	1.000	1.000
5	50	11	0	0.820	1.000	0.901
6	14	0	36	1.000	0.280	0.438
7	50	4	0	0.926	1.000	0.962
8	50	31	0	0.617	1.000	0.763
9	38	0	12	1.000	0.760	0.863
Average				0.932	0.848	0.856

Table 3: Per-digit precision, recall, and F1-score for the HMM.

Digit	TP	FP	FN	Precision	Recall	F1
0	40	0	10	1.000	0.800	0.889
1	44	8	6	0.846	0.880	0.863
2	26	0	24	1.000	0.520	0.684
3	43	2	7	0.956	0.860	0.905
4	37	0	13	1.000	0.740	0.851
5	46	24	4	0.657	0.920	0.767
6	13	0	37	1.000	0.260	0.413
7	48	1	2	0.980	0.960	0.970
8	50	42	0	0.543	1.000	0.704
9	22	0	28	1.000	0.440	0.611
Average				0.898	0.738	0.766

5. Analysis and Insights

5.1. Where Both Models Struggle

Looking at the confusion matrices and per-digit tables, three digits are consistently problematic across both models.

- **Digit 6 is the most difficult.** Digit 6 is by far the hardest for both models. The GMM achieves only 28% recall (14/50), while the HMM achieves 26% (13/50). Most of the errors are misclassifications as digit 8 — the GMM misclassifies 31 instances and the HMM misclassifies 35 instances of *six* as *eight*. This occurs because *six* and

eight share very similar vowel sounds and endings, and Speaker 6's pronunciation overlaps significantly with what the models learned for digit 8 during training.

- **Digit 2 is the second hardest.** Both models achieve only around 52–56% recall for digit 2. The GMM primarily confuses it with digit 0 (13 instances), while the HMM confuses it with digit 7 (18 instances). This is likely because the words *two*, *zero*, and *seven* share similar acoustic characteristics in their vowel regions, leading to ambiguity in MFCC features without phoneme-level modeling.
- **Digit 9 is particularly difficult for HMM.** The HMM struggles significantly with digit 9, misclassifying 23 out of 50 instances as digit 5. In contrast, the GMM performs better, correctly classifying 38 out of 50 instances. This confusion arises because *nine* and *five* have similar rhyming endings, which produce overlapping MFCC patterns, especially in the final frames of the utterance.
- **Digit 8 has high recall but low precision.** Digit 8 achieves perfect or near-perfect recall in both models. However, its precision is relatively low because it absorbs a large number of false positives from digit 6. As a result, the precision drops to 0.617 for the GMM and 0.543 for the HMM.

5.2. Validation vs Test Gap

There is also a considerable difference in performance in GMM between validation accuracy (96.80%) and testing accuracy (84.80%). It indicates that despite applying CMVN, the Gaussian parameters still incorporate certain speaker-dependent features of Speakers 1–5. If Speaker 6's frames lie in somewhat different areas of the feature space, the classifier makes a mistake.

For the HMM there is also an accuracy gap (87.47% for validation and 73.80% for testing), however, it generalizes more poorly. Being intrinsically speaker-independent by design, HMM with discretized emissions lacks details to tell one digit from another.

5.3. GMM vs HMM

The GMM is clearly the better model for this task because it did 11 percentage better than the HMM on the test set. The GMM works by treating each utterance as a group of independent frames and figuring out which digit's frame distribution best fits the data.

I noticed that this is a strong method for recognizing single digits because each digit usually takes up a different area in the 39-dimensional MFCC feature space. The GMM can effectively capture the multi-modal nature of these frame distributions by using 16 mixture components per class, without needing to explicitly model temporal structure.

The HMM is theoretically more principled because it models the sequential structure of speech. However, the VQ step limits its performance in practice. When you change

39-dimensional continuous features into 128 discrete symbols, you lose information. This makes the emission estimates noisy when you don't have much training data. In this case, the loss of data is greater than the benefits of temporal modeling.

In real-world large-vocabulary speech recognition, HMMs almost always do better than GMMs. This is because temporal structure is very important when recognizing thousands of words instead of just 10 separate digits. In that case, HMMs are also used with a continuous emission model (GMM-HMM) instead of a discrete one. This completely avoids the problem of quantization loss.

6. Conclusion

- This assignment involved building a digit recogniser from scratch and evaluating its cross-speaker performance using a held-out speaker (Speaker 6).
- Two models were implemented and compared: a Gaussian Mixture Model (GMM) and a Discrete Hidden Markov Model (HMM) with Vector Quantization (VQ).
- The GMM achieved 84.80% test accuracy on Speaker 6, while the HMM achieved 73.80%.
- The GMM performs better for this task because its frame-level modeling with 16 diagonal Gaussian components per digit captures sufficient discriminative information without suffering from the quantization loss that limits the discrete HMM.
- The most impactful design choices were the use of 39-dimensional MFCC $+\Delta + \Delta\Delta$ features and CMVN normalisation, which significantly improved cross-speaker generalisation.
- The hardest digits to classify were:
 - Digit 6: often confused with digit 8
 - Digit 2: often confused with digits 0 and 7
 - Digit 9: often confused with digit 5

****Instruction to Run Codes:**** Make a folder for all recordings and upload the 4 code files (feature-extraction,gmm,hmm and main) and run all of them.